



LISTA DE EXERCÍCIOS – FUNDAMENTOS DE SISTEMAS INTELIGENTES

Lista de Exercícios 1

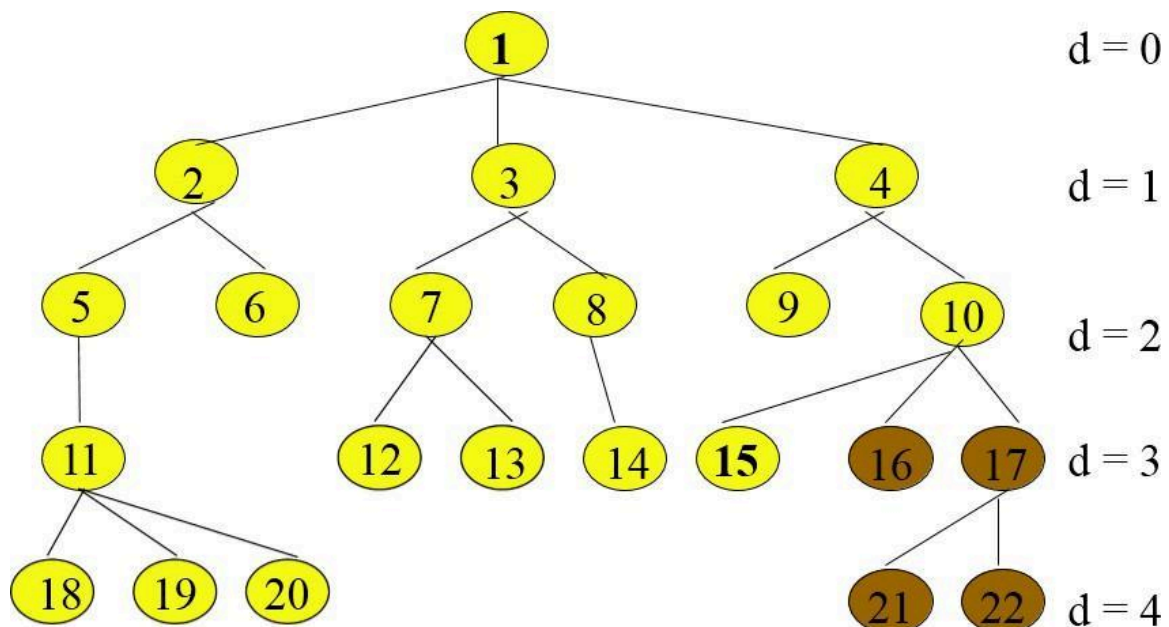
DOCENTE: Thiago França Naves

DATA: ____ / ____ / ____

ALUNO: Leonardo Jose Reis Pinto

OBS: Durante o período de APNP todos os exercícios resolvidos devem conter além do desenvolvimento da técnica ou do algoritmo de resposta, também um resumo de como você chegou na solução, suas partes e relação com o algoritmo/técnica desenvolvido. Esse será um dos principais critérios de avaliação da resposta e atribuição de nota a lista.

1. Simule busca em largura e em profundidade no cenário de problema modelado pela árvore abaixo para encontrar os nós 10 e 20. Compare tanto o número de nós visitados quanto expandidos, por fim diga qual o uso de tempo e memória de cada algoritmo na sua execução.



Resultado: Desenvolvi um script em python para resolver o problema, então segue o script e o output:

Leo@Leo-ASUS-TUF-Gaming-A16-FA607NUG-FA607NUG:

balho/UTFPR/Fundamentos de IA/Listas de exerci

BUSCA EM LARGURA

Alvo 10:

Encontrado nó: 10

Caminho: [1, 4, 10]

Nós visitados: 10

Nós expandidos: 9

Tempo de execução: 0.000009 segundos

Alvo 20:

Encontrado nó: 20

Caminho: [1, 2, 5, 11, 20]

Nós visitados: 20

Nós expandidos: 19

Tempo de execução: 0.000007 segundos

BUSCA EM PROFUNDIDADE (DFS)

Alvo 10:

Encontrado nó: 10

Caminho: [1, 4, 10]

Nós visitados: 17

Nós expandidos: 16

Tempo de execução: 0.000011 segundos

Alvo 20:

Encontrado nó: 20

Caminho: [1, 2, 5, 11, 20]

Nós visitados: 7

Nós expandidos: 6

Tempo de execução: 0.000003 segundos

```
def dfs(raiz, alvo):
    visitados = 0
    expandidos = 0
    pilha = [(raiz, [raiz.valor])]
    conjunto_visitados = set()
    inicio = time.perf_counter()

    while pilha:
        atual, caminho = pilha.pop()

        if atual in conjunto_visitados:
            continue

        conjunto_visitados.add(atual)
        visitados += 1

        if atual.valor == alvo:
            fim = time.perf_counter()
            return {
                'encontrado': atual.valor,
                'caminho': caminho,
                'visitados': visitados,
                'expandidos': expandidos,
                'tempo': fim - inicio
            }

        expandidos += 1
        for filho in reversed(atual.filhos):
            if filho not in conjunto_visitados:
                pilha.append((filho, caminho + [filho.valor]))

    fim = time.perf_counter()
    return {'encontrado': None, 'caminho': [], 'visitados': visitados,
            'expandidos': expandidos, 'tempo': fim - inicio}
```

```

def bfs(raiz, alvo):
    visitados = 0
    expandidos = 0
    fila = deque([(raiz, [raiz.valor])])
    conjunto_visitados = set()
    inicio = time.perf_counter()

    while fila:
        atual, caminho = fila.popleft()

        if atual in conjunto_visitados:
            continue

        conjunto_visitados.add(atual)
        visitados += 1

        if atual.valor == alvo:
            fim = time.perf_counter()
            return {
                'encontrado': atual.valor,
                'caminho': caminho,
                'visitados': visitados,
                'expandidos': expandidos,
                'tempo': fim - inicio
            }

        expandidos += 1
        for filho in atual.filhos:
            if filho not in conjunto_visitados:
                fila.append((filho, caminho + [filho.valor]))

    fim = time.perf_counter()
    return {'encontrado': None, 'caminho': [], 'visitados': visitados,
            'expandidos': expandidos, 'tempo': fim - inicio}

```

2. Defina com suas próprias palavras os seguintes termos:

a. Estado

R: Um estado representa a situação atual do problema em um determinado instante, contendo todas as informações relevantes para que o algoritmo possa tomar decisões. Por exemplo, em um labirinto, o estado é a posição atual do agente.

b. Espaço de estados

R: É o conjunto de todos os estados possíveis que o problema pode assumir, desde o estado inicial até o estado objetivo. O algoritmo de busca navega por esse espaço tentando encontrar um caminho até o objetivo. É basicamente o "universo" de configurações que o problema pode ter.

c. Árvore de busca

R: É uma estrutura hierárquica usada para representar a exploração do espaço de estados. A raiz é o estado inicial, cada nó é um estado visitado, e os ramos são as ações que levam de um estado ao próximo. Diferente do grafo de estados, a árvore pode ter o mesmo estado representado em múltiplos nós.

d. Nó de busca

R: É cada elemento da árvore de busca. Ele armazena o estado correspondente, um ponteiro para o nó pai, a ação que gerou esse nó e o custo acumulado do caminho desde a raiz até ele.

e. Estado objetivo

R: É o estado (ou conjunto de estados) que o agente deseja alcançar. Funciona como a condição de parada do algoritmo — quando o estado atual satisfaz o objetivo, a busca termina.

f. Ação

R: É uma operação que pode ser aplicada a um estado para gerar um novo estado (sucessor). Cada ação representa um possível "passo" na resolução do problema e normalmente possui um custo associado.

g. Função-sucessor

R: Dado um estado, retorna todos os estados que podem ser alcançados a partir dele por meio das ações disponíveis. É ela que "gera os filhos" de um nó na árvore de busca, definindo como o espaço de estados é explorado.

h. Fator de branching

R: É o número médio de filhos (sucessores) de cada nó na árvore de busca. Um fator de branching alto significa que cada estado gera muitos sucessores, o que aumenta exponencialmente o tamanho da árvore e torna a busca mais custosa.

i. Custo do caminho

R: É a soma dos custos de todas as ações realizadas desde o estado inicial até o estado atual. Representa o "preço total" para se chegar a determinado estado a partir do início.

j. Solução do problema

R: É a sequência de ações que leva do estado inicial ao estado objetivo. Uma solução ótima é aquela com o menor custo de caminho dentre todas as soluções possíveis.

3. Descreva o estado inicial, a função de teste, a função-sucessor e a função de custo para cada um dos seguintes problemas. Modele como algoritmos de busca poderiam resolver cada um destes problemas exemplificando como seria um estado, como são gerados novos estados e quando o algoritmo poderia encontrar a solução, utilize representações gráficas e desenhos para esboçar a modelagem e execução do algoritmo.

- a. Você tem que colorir um mapa plano usando somente 4 cores, de tal forma que duas regiões adjacentes não tenham a mesma cor.

R:

ESTADO INICIAL= O mapa está em branco, sem nenhuma cor definida

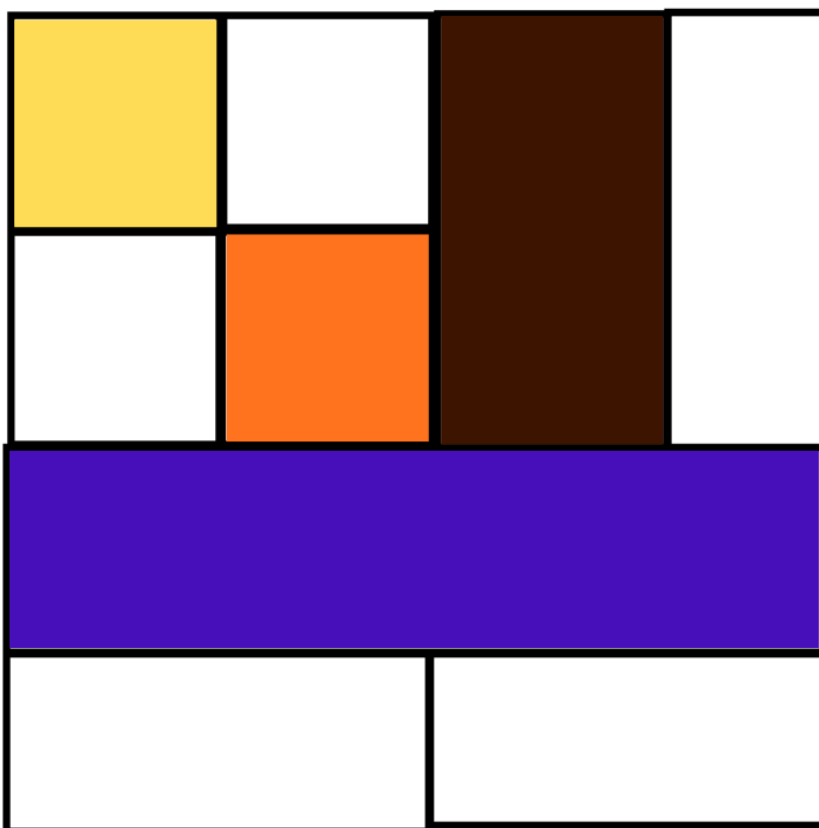
FUNÇÃO DE OBJETIVO= Todas as áreas do mapa devem ser coloridas

Porém nenhuma das áreas devem ter a mesma cor como as vizinhas.

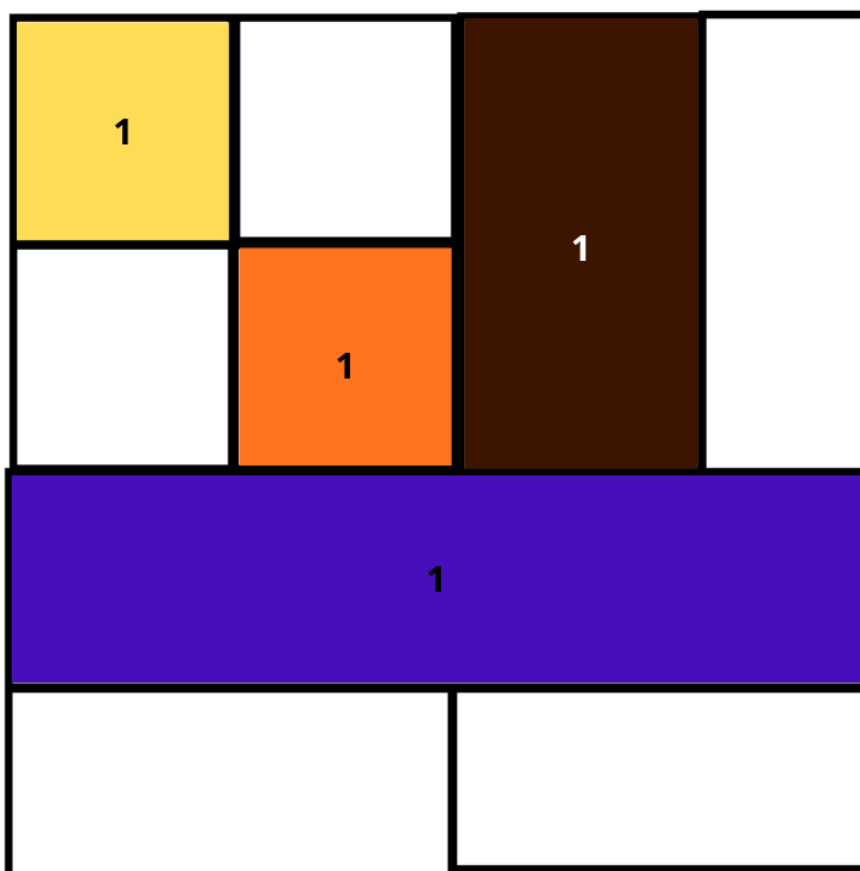
FUNÇÃO SUCESSOR= Para cada área que ainda não possui cor, deverá ser atribuída uma cor das disponíveis.

FUNÇÃO DE CUSTO = Ao atribuir uma cor para a área que é válida deverá ser atribuído um custo uniforme para a área de 1.

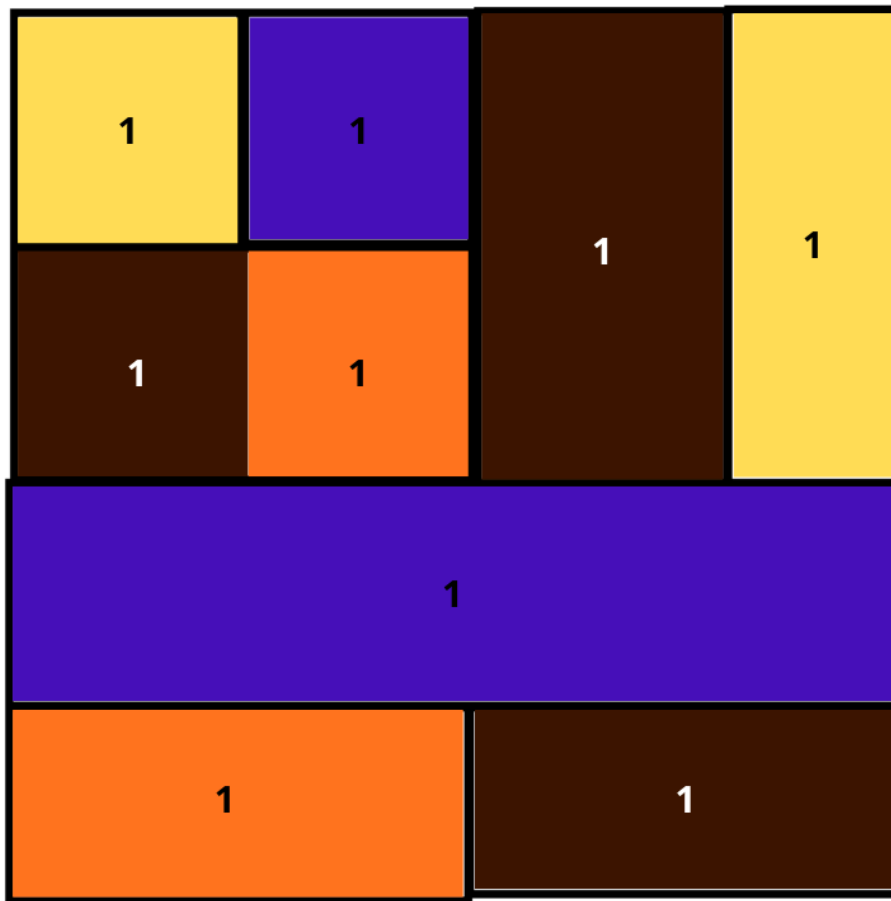
O estado Inicial do mapa:



A função sucessor irá pintar as áreas que não possuem cor:



A função objetiva com todas as áreas do mapa coloridas e com peso:



- b. Você tem três jarros, medindo 12 litros, 8 litros e 3 litros e uma fonte de água. Você pode encher ou esvaziar os jarros de um para o outro ou no chão. Você quer medir exatamente um litro em qualquer jarro.

R: ESTADO INICIAL= A descrição do cenário do enunciado como está na imagem abaixo.

FUNÇÃO DE OBJETIVO= O macaco conseguir pegar a banana.

FUNÇÃO SUCESSOR= Mover os caixotes, empilhar os caixotes, subir nos caixotes

FUNÇÃO DE CUSTO = Cada ação será designada um peso 1.

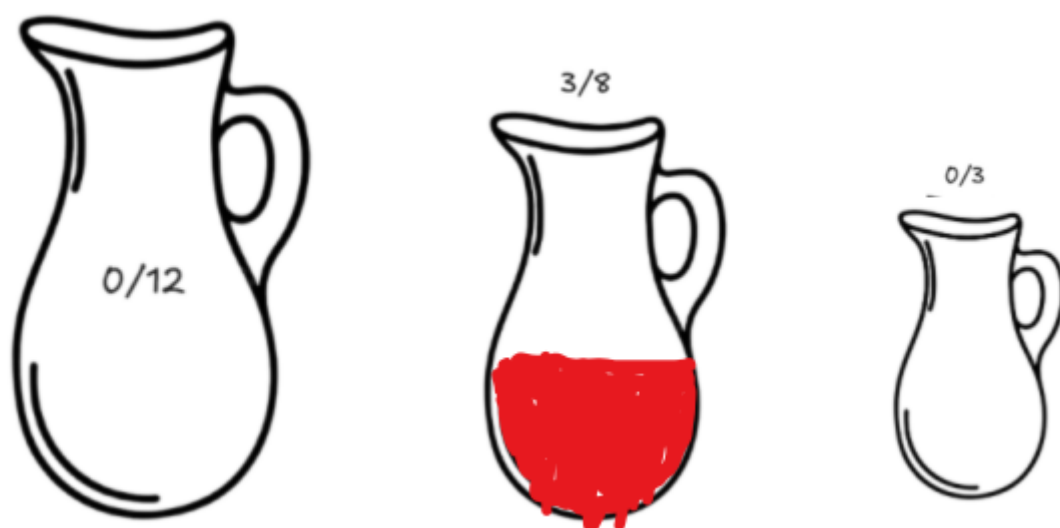
Estado inicial do problema:



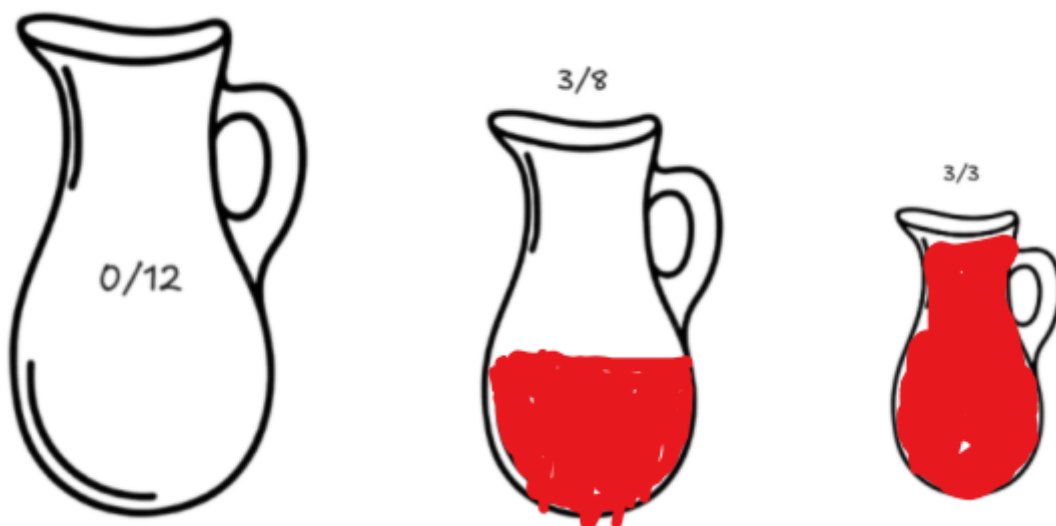
Função sucessora, encher o jarro de 3 litros custo total = 1:



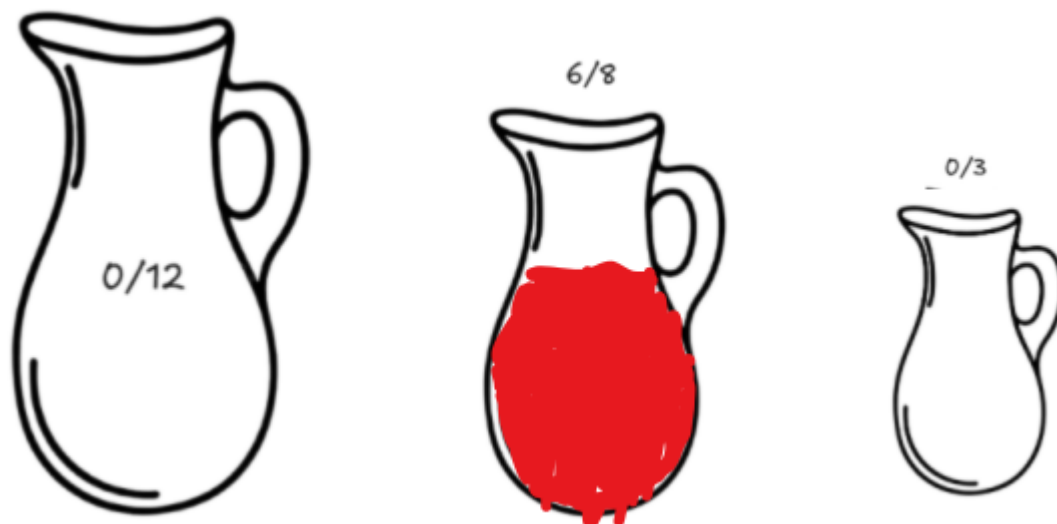
Função sucessora, transferir do jarro de 3 litros para o de 8 custo total = 1:



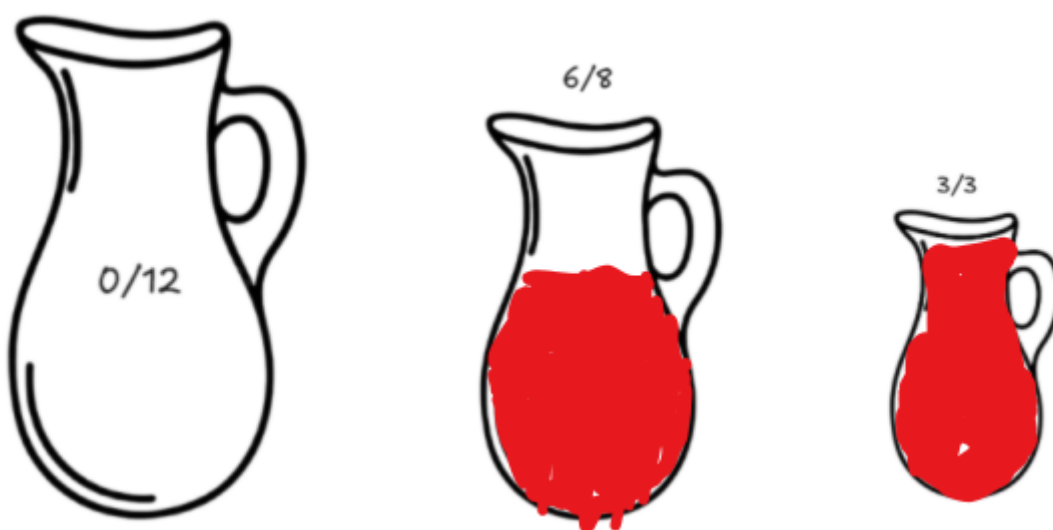
Função sucessora, encher o jarro de 3 litros custo total = 1:



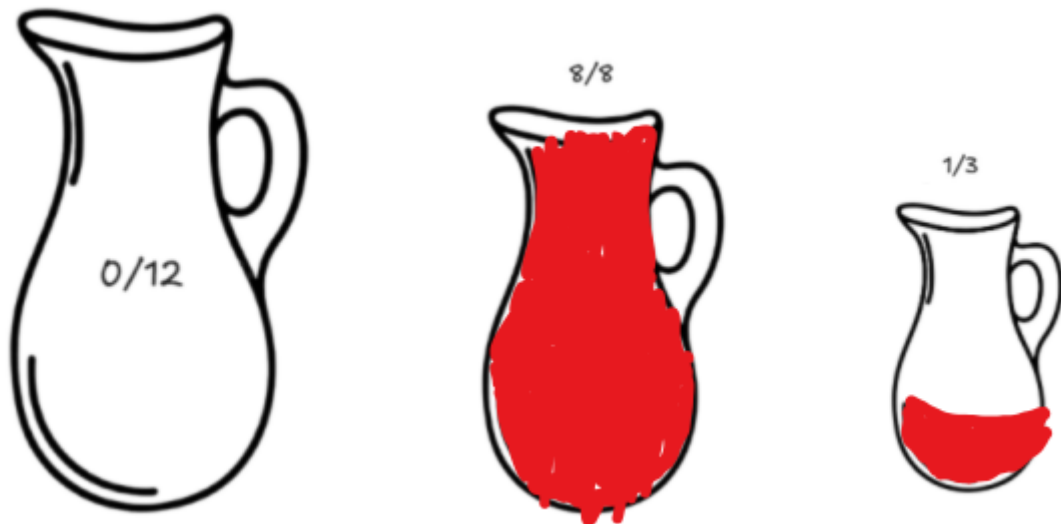
Função sucessora, transferir do jarro de 3 litros para o de 8 custo total = 1:



Função sucessora, encher o jarro de 3 litros custo total = 1:



Função sucessora, transferir do jarro de 3 litros para o de 8 custo total = 1:
Função objetivo, jarro 3 com 1 litro:



- c. Um macaco de meio metro de altura está em uma jaula onde algumas bananas estão suspensas a três metros e meio do chão. Ele quer pegar bananas. A jaula contém dois caixotes de um metro e meio cada, que podem ser movidos e sobrepostos.

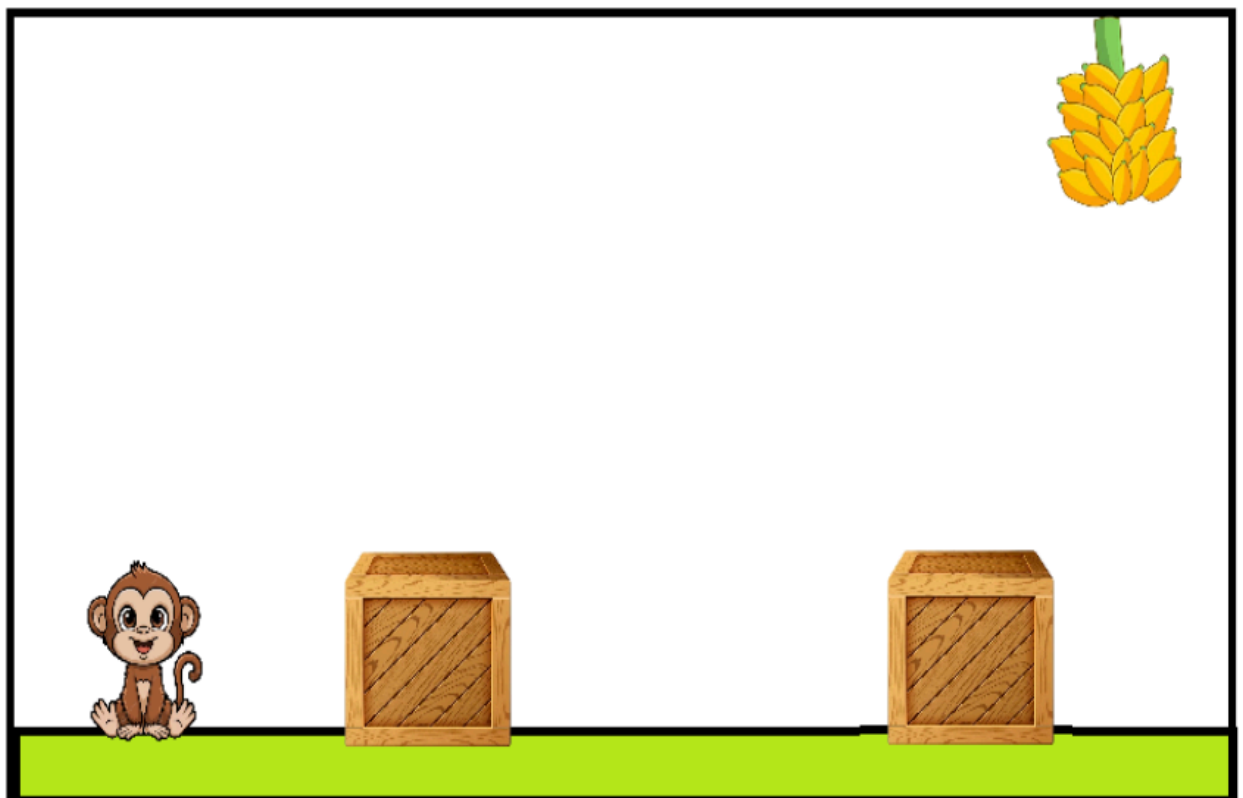
R: ESTADO INICIAL= A descrição do cenário do enunciado como está na imagem abaixo.

FUNÇÃO DE TESTE OBJETIVO = O macaco conseguir pegar a banana.

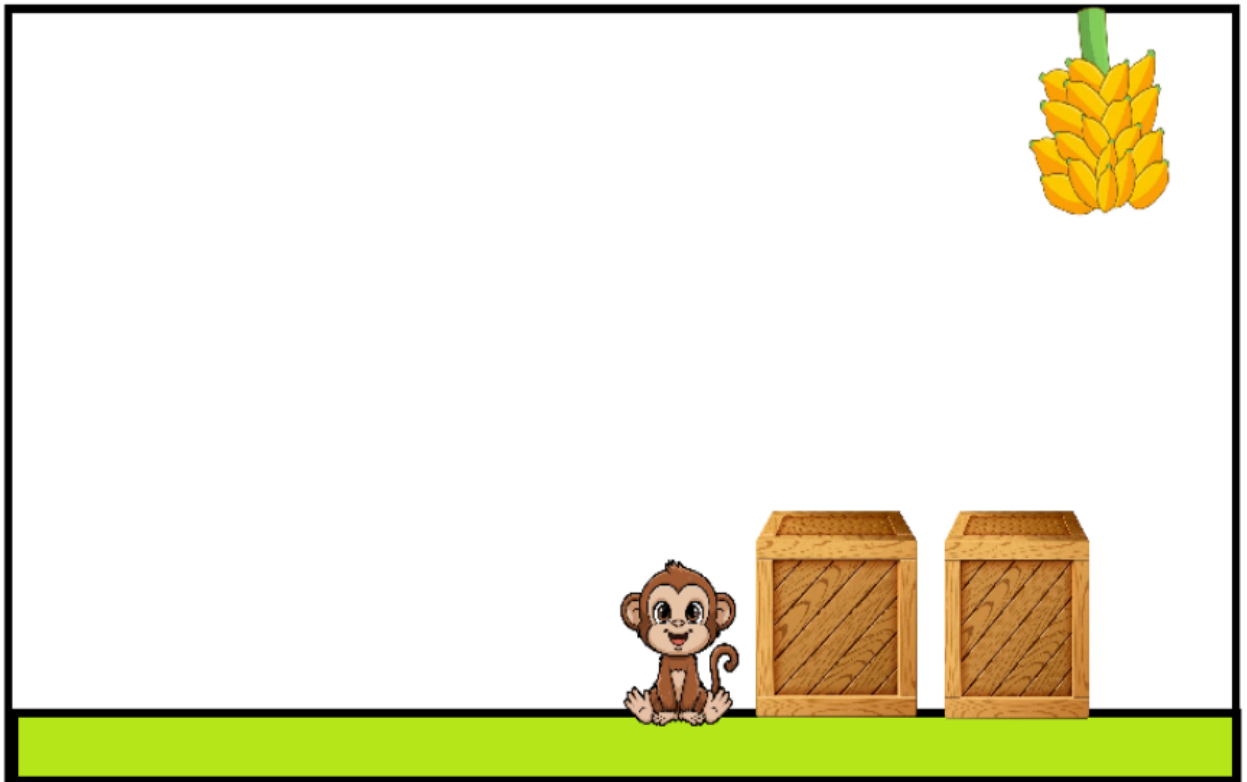
FUNÇÃO SUCESSOR= Mover os caixotes, empilhar os caixotes, subir nos caixotes

FUNÇÃO DE CUSTO = Cada ação será designada um peso 1.

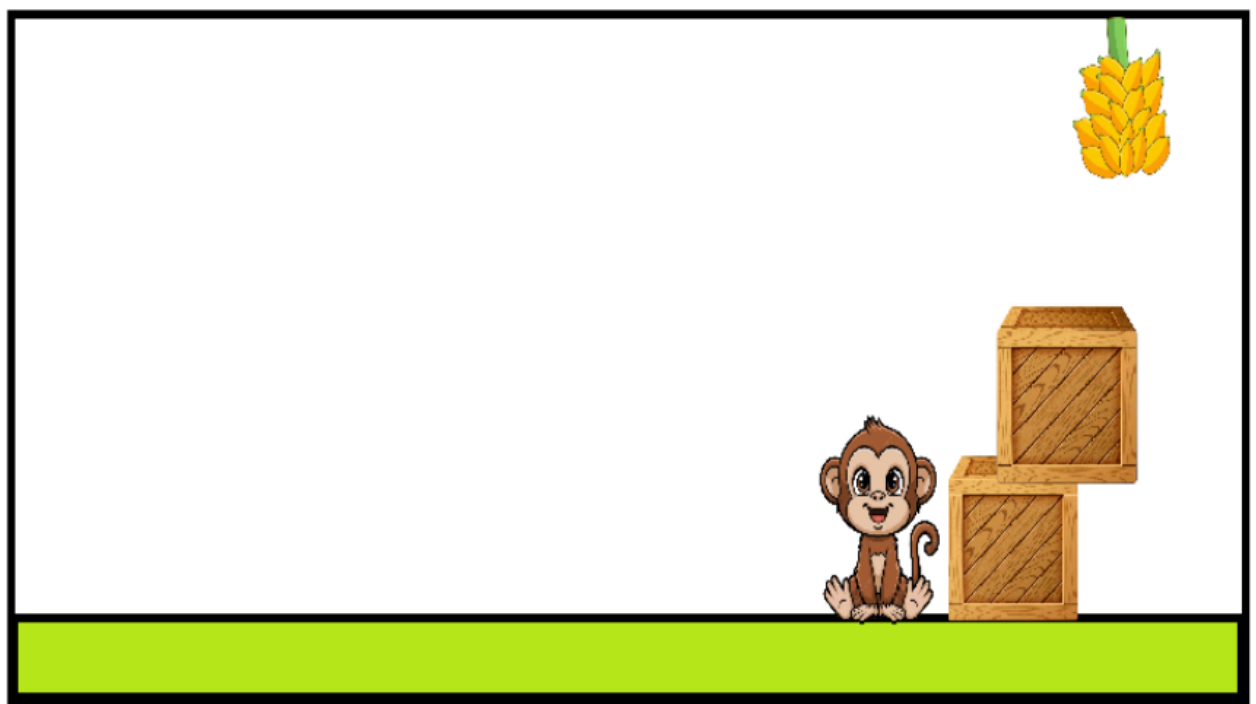
Estado inicial do problema:



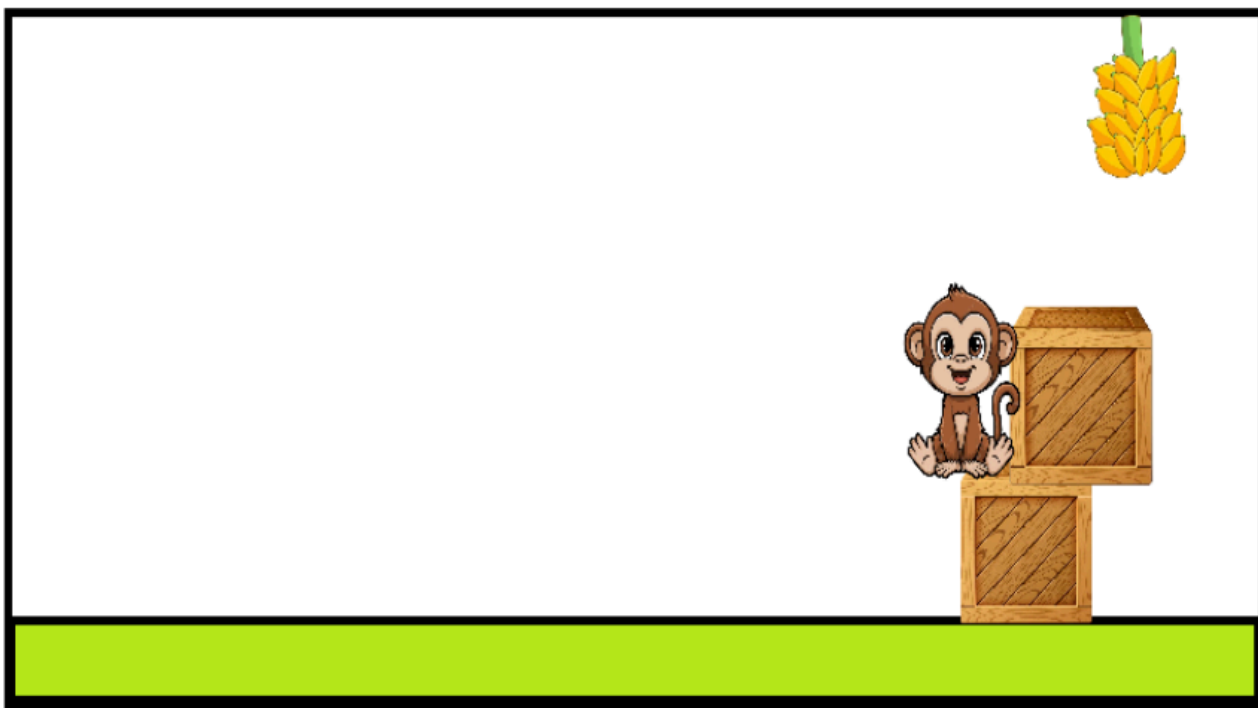
Move o caixote, custo = 1



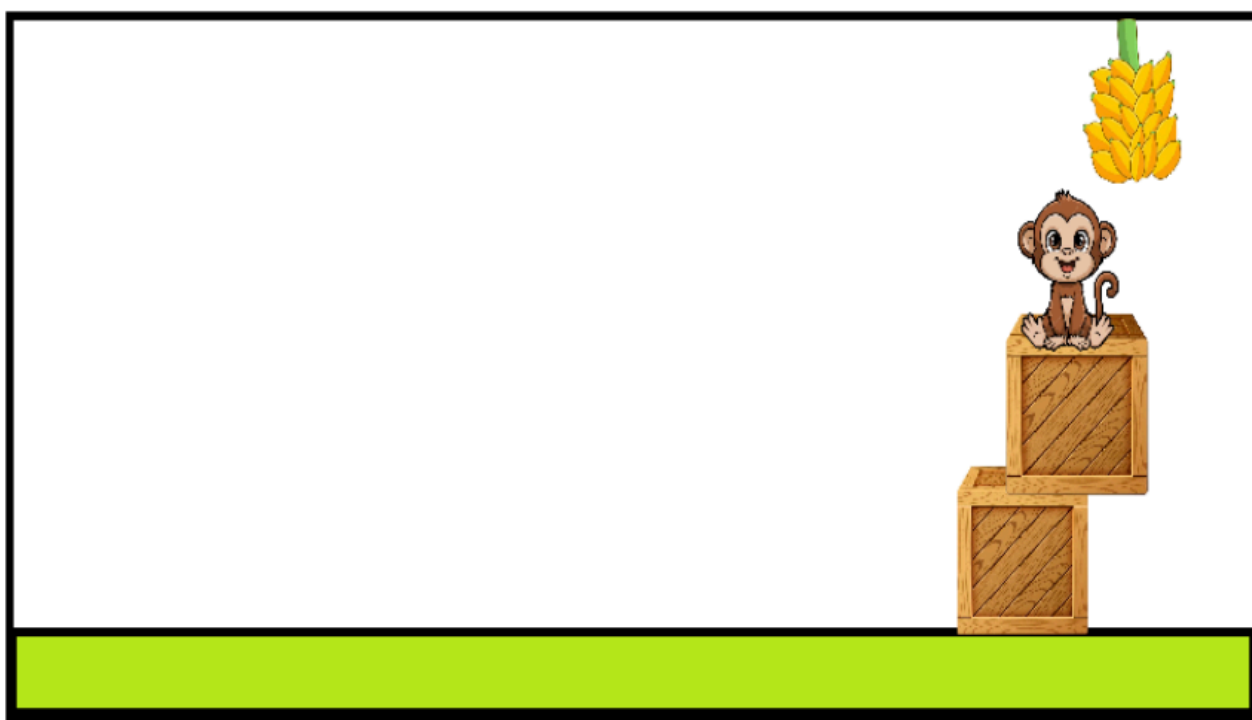
Empilha caixote, custo = 2



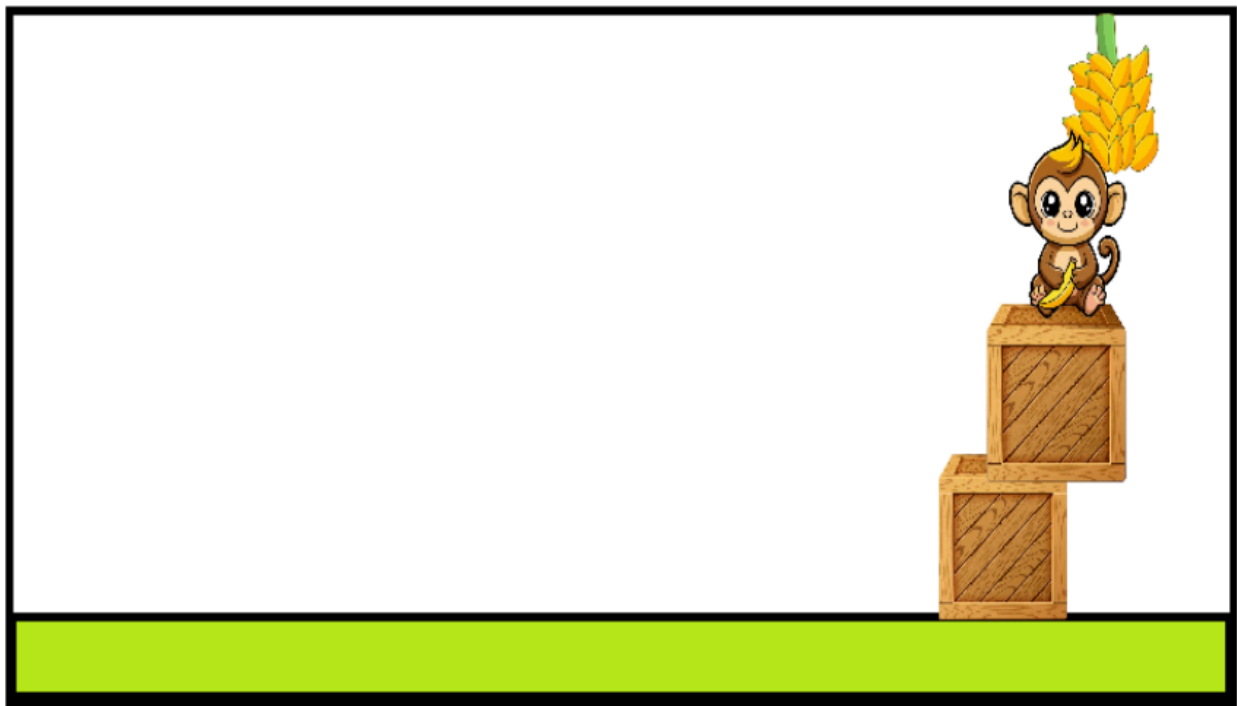
Sobre no caixote 1, custo = 3



Sobre no caixote 2, custo= 4



Pega a banana, custo = 5



4. O que difere uma estratégia de busca A de uma estratégia de busca B e como se avalia geralmente as estratégias de busca (critérios)?

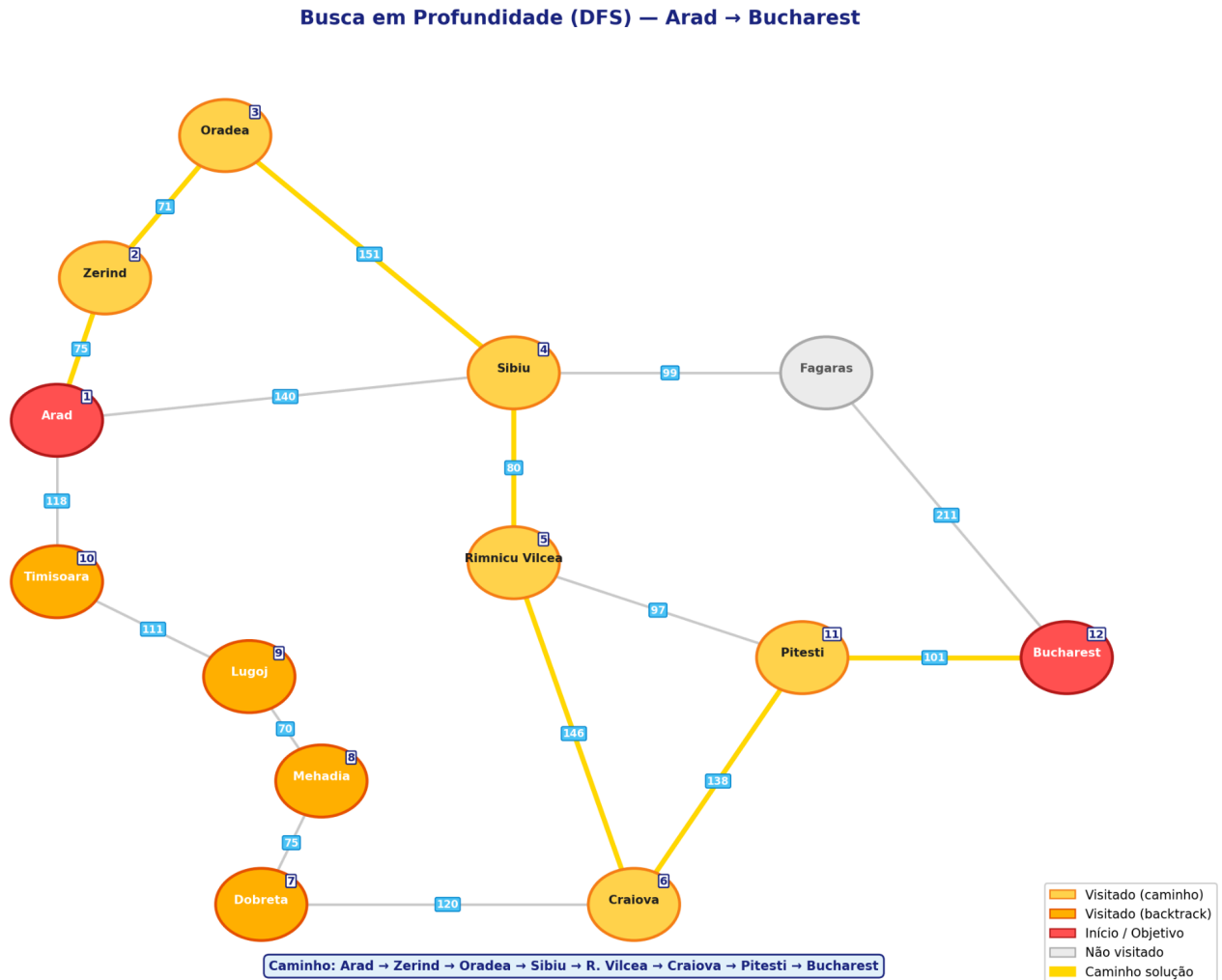
R: O que muda de uma estratégia pra outra é basicamente a ordem em que os nós da fronteira são expandidos. A fronteira nada mais é que o conjunto de nós que já foram gerados mas ainda não foram explorados, e cada estratégia tem um critério próprio para escolher qual expandir primeiro, podendo ser o mais raso, o mais profundo, o de menor custo, etc. Para comparar as estratégias a gente normalmente usa 4 critérios: completude, que diz se o algoritmo sempre acha uma solução caso ela exista; otimalidade, que diz se a solução encontrada é sempre a de menor custo; complexidade de tempo, que mede quantos nós são gerados ou expandidos até achar a solução, geralmente em função do fator de ramificação b e da profundidade de complexidade de espaço, que mede quanta memória o algoritmo precisa durante a execução. Dependendo do problema uma estratégia acaba sendo melhor que a outra. Por exemplo, num espaço bem profundo mas com solução rasa a BFS acha rápido, enquanto num espaço largo com solução profunda a DFS gasta bem menos memória.

5. Explique rapidamente cada uma das estratégias abaixo, destacando qual nó da fronteira é visitado primeiro, como a fronteira se comporta e o desempenho para cada uma delas. Em seguida simule cada um deles para o problema do mapa da Romênia (de Arad a Bucareste).
- Busca em profundidade

R: A DFS explora um caminho até a maior profundidade possível antes de retroceder, ou seja, faz backtracking. Ela usa uma pilha LIFO, então o último nó inserido na fronteira é sempre o próximo a ser expandido, e a própria fronteira é mantida como essa pilha. Em termos de desempenho, ela não é completa em grafos com ciclos porque pode entrar em loop infinito, e também não é ótima, já que pode achar uma solução mais longa antes de uma mais curta. Tempo é $O(b^m)$ e memória é $O(b \cdot m)$, com m sendo a profundidade máxima. Simulando de Arad até Bucharest, a DFS segue sempre o primeiro vizinho disponível: parte de Arad e visita Zerind, depois Oradea, Sibiu, Rimnicu Vilcea, Craiova, Dobreta, Mehadia, Lugoj e Timisoara; nesse ponto Timisoara não tem mais vizinhos não visitados, então rola um backtrack até

Craiova, que visita Pitesti, e Pitesti finalmente chega em Bucharest.

Caminho encontrado: Arad -> Zerind -> Oradea -> Sibiu -> Rimnicu Vilcea -> Craiova -> Pitesti -> Bucharest.

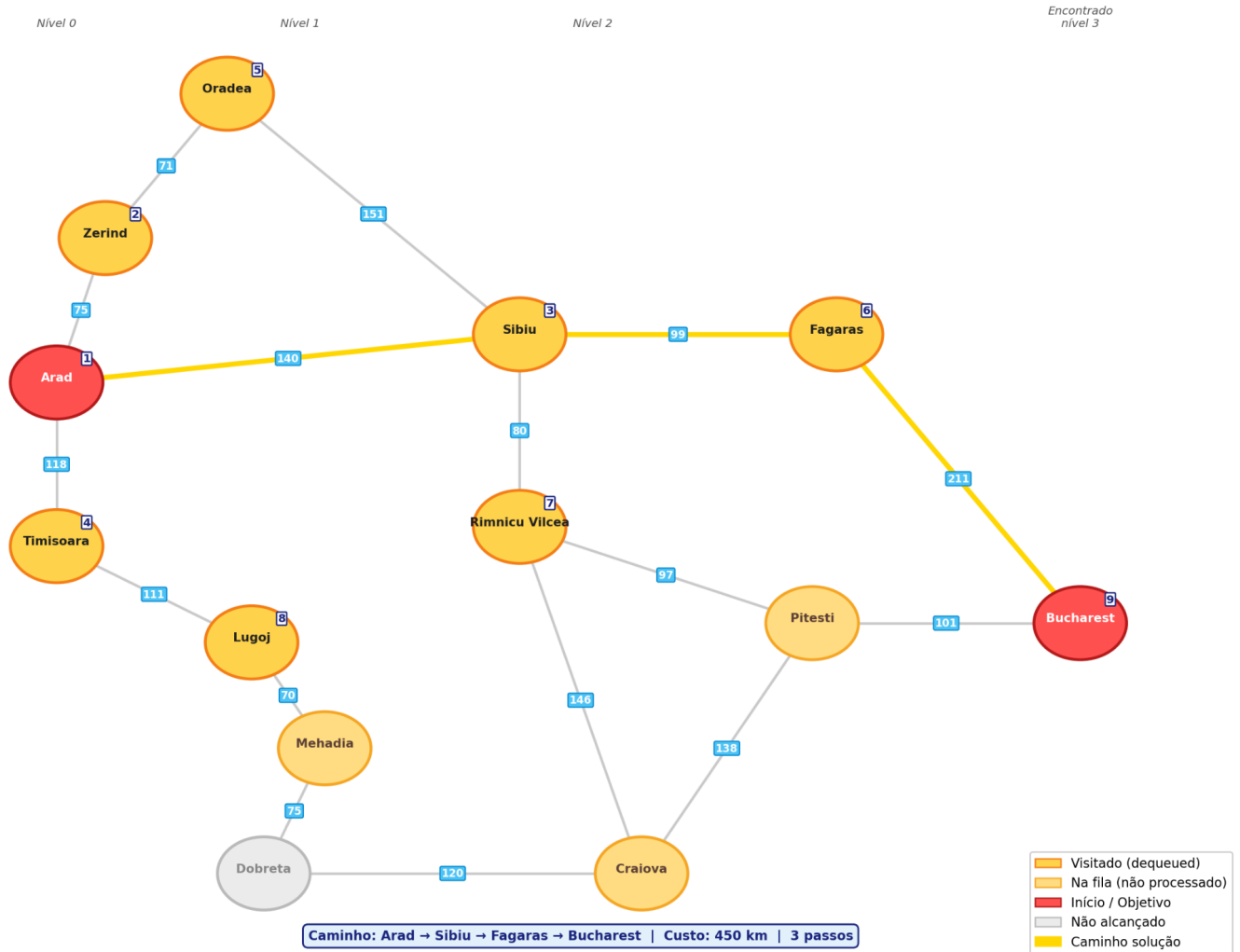


b. Busca em extensão (ou largura)

R: A BFS funciona explorando os nós em camadas, ou seja, primeiro visita todos os nós a uma distância 1 do inicial, depois todos a distância 2, e assim por diante, até achar o objetivo. Ela usa uma fila FIFO, então o primeiro nó inserido na fronteira é sempre o primeiro a ser expandido, o que garante que quando o objetivo for encontrado ele vai estar no menor número de passos, ou seja, a solução é ótima em termos de quantidade de arestas. Simulando de Arad até Bucareste, a fronteira começa como [Arad]; expandindo Arad, entram seus vizinhos e a fronteira fica [Sibiu, Timisoara, Zerind]; expandindo Sibiu, entram Fagaras, Rimnicu Vilcea e Oradea, deixando a fronteira como [Timisoara, Zerind, Fagaras, Rimnicu Vilcea, Oradea]; expandindo Timisoara, entra Lugoj e a fronteira fica [Zerind, Fagaras, Rimnicu Vilcea, Oradea, Lugoj]; expandindo Zerind, ele tentaria adicionar Oradea, mas como já está na fronteira é ignorado; aí expandindo Fagaras, entra Bucareste e o objetivo é encontrado. Esse caminho tem 3 arestas, que é o mínimo possível, e a BFS garante que não existiria outro menor.

Caminho encontrado: Arad -> Sibiu -> Fagaras -> Bucareste.

Busca em Largura (BFS) — Arad → Bucharest

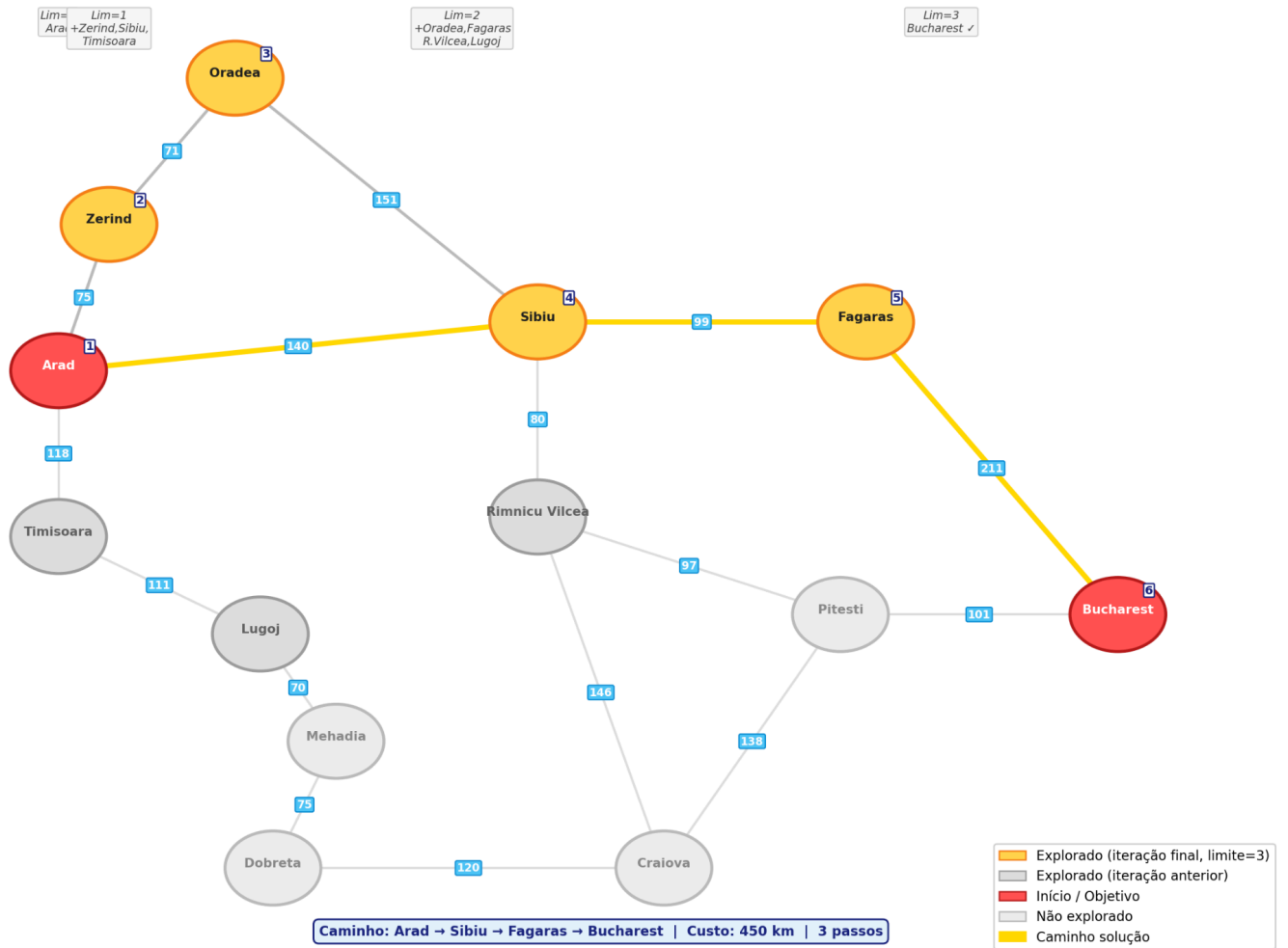


c. Busca em profundidade iterativa

R: A BFS explora todos os nós de um nível antes de passar pro próximo. Ela usa uma fila FIFO, então o primeiro nó inserido é o primeiro a ser expandido, e isso garante que o caminho encontrado tem o menor número de passos. Em termos de desempenho ela é completa e ótima quando todos os custos são iguais, e tanto o tempo quanto a memória são $O(b^d)$, onde d é a profundidade da solução mais rasa. Simulando de Arad até Bucharest, a fronteira começa como [Arad]; expandindo Arad, entram seus vizinhos e a fronteira fica [Zerind, Sibiu, Timisoara]; expandindo Zerind, entra Oradea e a fronteira vira [Sibiu, Timisoara, Oradea]; expandindo Sibiu, entram Fagaras e Rimnicu Vilcea; expandindo Timisoara, entra Lugoj; expandindo Oradea, Zerind e Sibiu já tinham sido visitados, então nada é adicionado; e por fim expandindo Fagaras, entra Bucharest e o objetivo é encontrado.

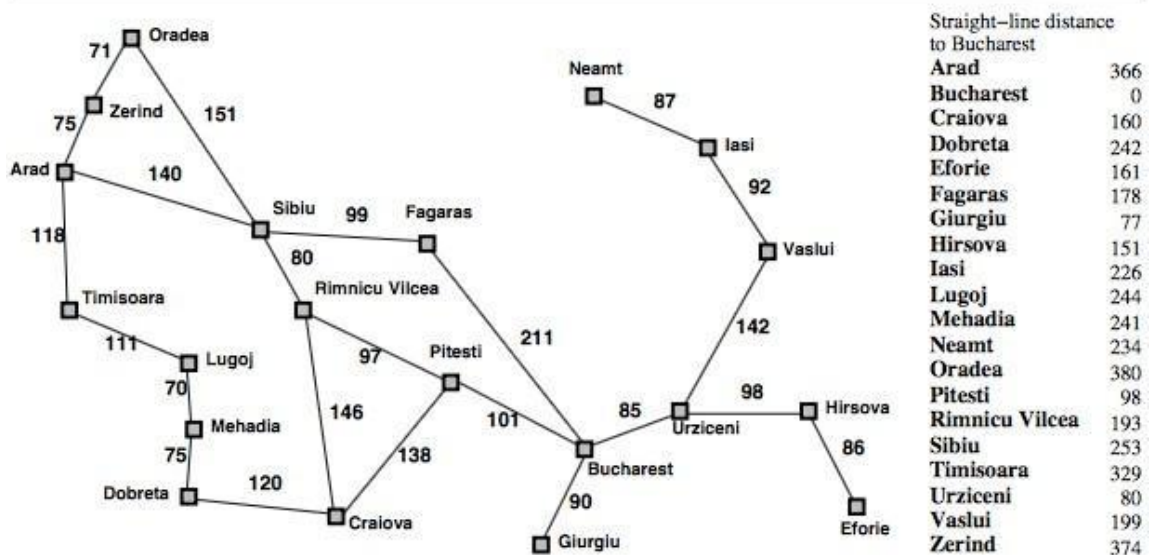
Caminho encontrado: Arad → Sibiu → Fagaras → Bucharest.

Busca em Profundidade Iterativa (IDS) — Arad → Bucharest (Iteração final: limite = 3)



Caminho encontrado: Arad -> Sibiu -> Fagaras -> Bucharest

Romênia com custos por passo em km



6. Discorra sobre completude, otimalidade, custo de tempo e memória dos algoritmos Busca em Largura, Profundidade, Profundidade Limitada e Aprofundamento Iterativo. Existe alguma condição para que as estratégias sejam ótimas ou completas.

R:

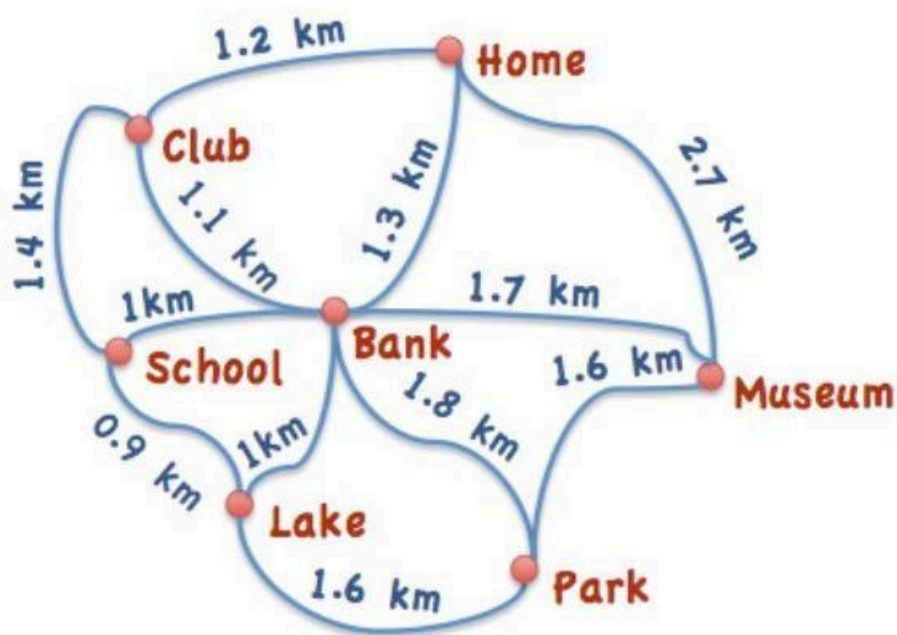
A BFS é completa e ótima quando os custos das arestas são iguais, já que explora nível por nível e a primeira solução achada tem o menor número de passos. O problema é que ela guarda todos os nós do nível atual em memória, então tempo e espaço são $O(b^d)$, o que escala mal e inviabiliza problemas grandes.

A DFS vai no extremo oposto: não é completa em grafos com ciclos e nem é ótima, mas gasta bem menos memória, $O(b \cdot m)$, porque só mantém o caminho atual na pilha. O tempo no pior caso é $O(b^m)$, que pode ser alto se a árvore for muito funda.

A DLS tenta resolver o loop infinito da DFS colocando um limite l de profundidade, mas só acha solução se $l \geq d$, então não é completa em geral, e também não é ótima. Tempo $O(b^l)$ e memória $O(b \cdot l)$. A IDS pega essa ideia e roda a DLS com limites crescentes 0, 1, 2... até achar. Resultado: completa, ótima sob custo uniforme, tempo $O(b^d)$ e memória $O(b \cdot d)$. Na prática é a mais equilibrada das quatro, e a reexpansão de nós pesa pouco já que a maior parte deles fica nos níveis mais profundos.

No geral, pra ser ótima a estratégia precisa de custos uniformes, senão "menos passos" não significa "menor custo". E pra ser completa, ela precisa lidar com ciclos e ter espaço de busca finito. BFS e IDS atendem isso por design, a DFS quebra com ciclos e a DLS quebra se l for menor que a profundidade real.

7. Considerando o mapa abaixo:



Utilize os algoritmos abaixo para sair de "School" e chegar em "Museum". Qual dos algoritmos apresentou melhor resultado?

- a) Busca em Largura

R:

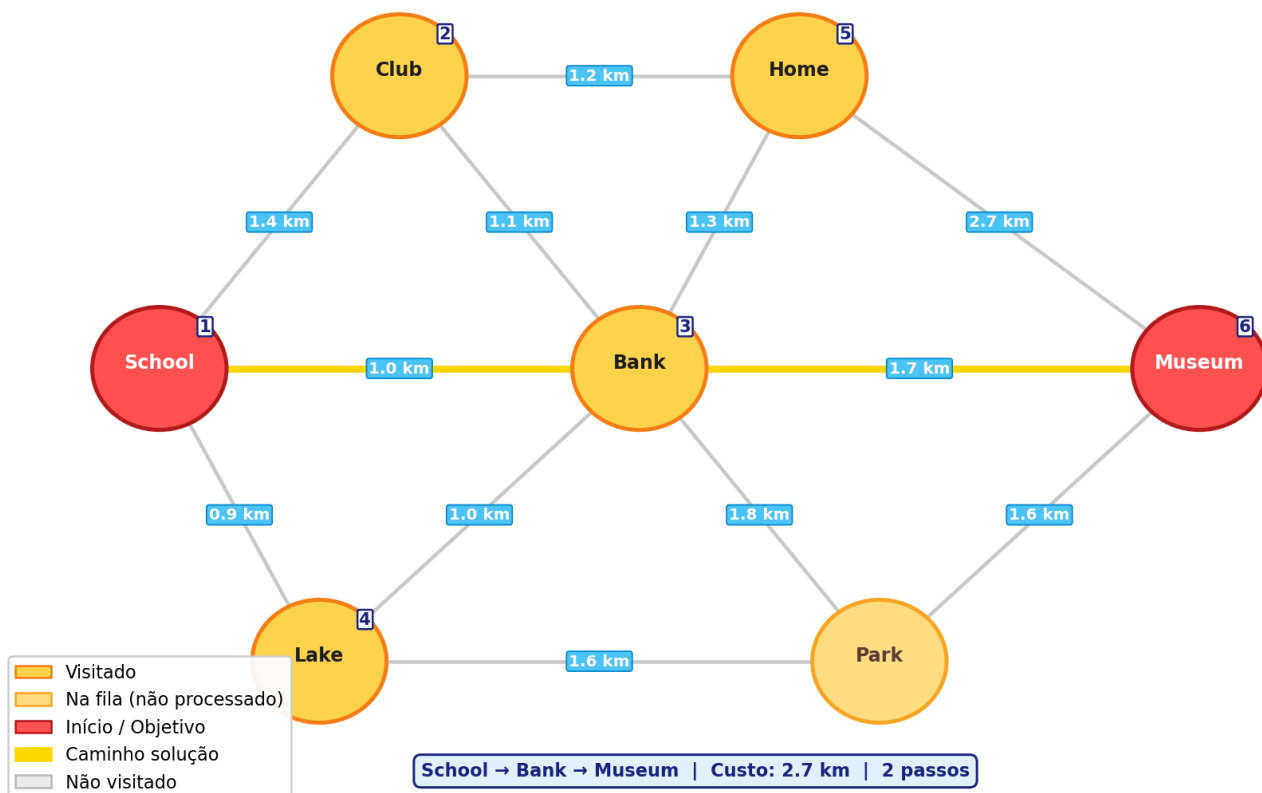
BFS explora camada por camada a partir de School:

1. Nível 0: School
2. Nível 1: Club, Bank, Lake
3. Nível 2: Home (de Club), Museum (de Bank)

Caminho: School -> Bank -> Museum | Custo: 2.7 km | Passos: 2

segue o exemplo:

Busca em Largura (BFS) — School → Museum



b) Profundidade

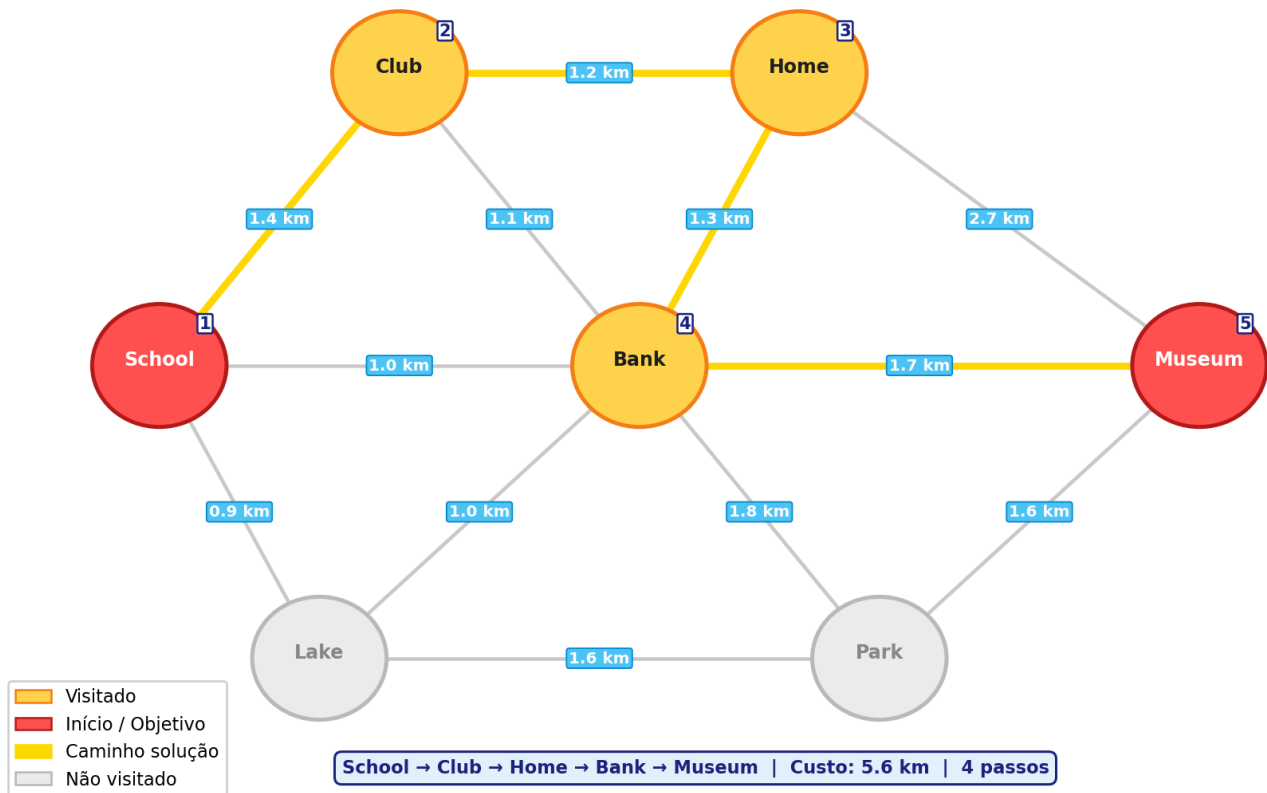
R: A DFS vai fundo pelo primeiro vizinho de School, então o resultado depende da ordem de exploração

Caminho: School -> Club -> Home -> Bank -> Museum Custo: 5.6 km Passos: 4

Vale notar que o resultado da DFS varia conforme a ordem dos vizinhos, e o caminho encontrado não é necessariamente o de menor custo

segue o exemplo:

Busca em Profundidade (DFS) — School → Museum



c) Profundidade Iterativa (com limite de 4)

R: No limite 0, só a School é visitada. No limite 1, ela alcança Club, Bank e Lake, mas nenhum deles é o Museum. No limite 2, expandindo Bank, o algoritmo chega em Museum em 2 passos, e a busca encerra

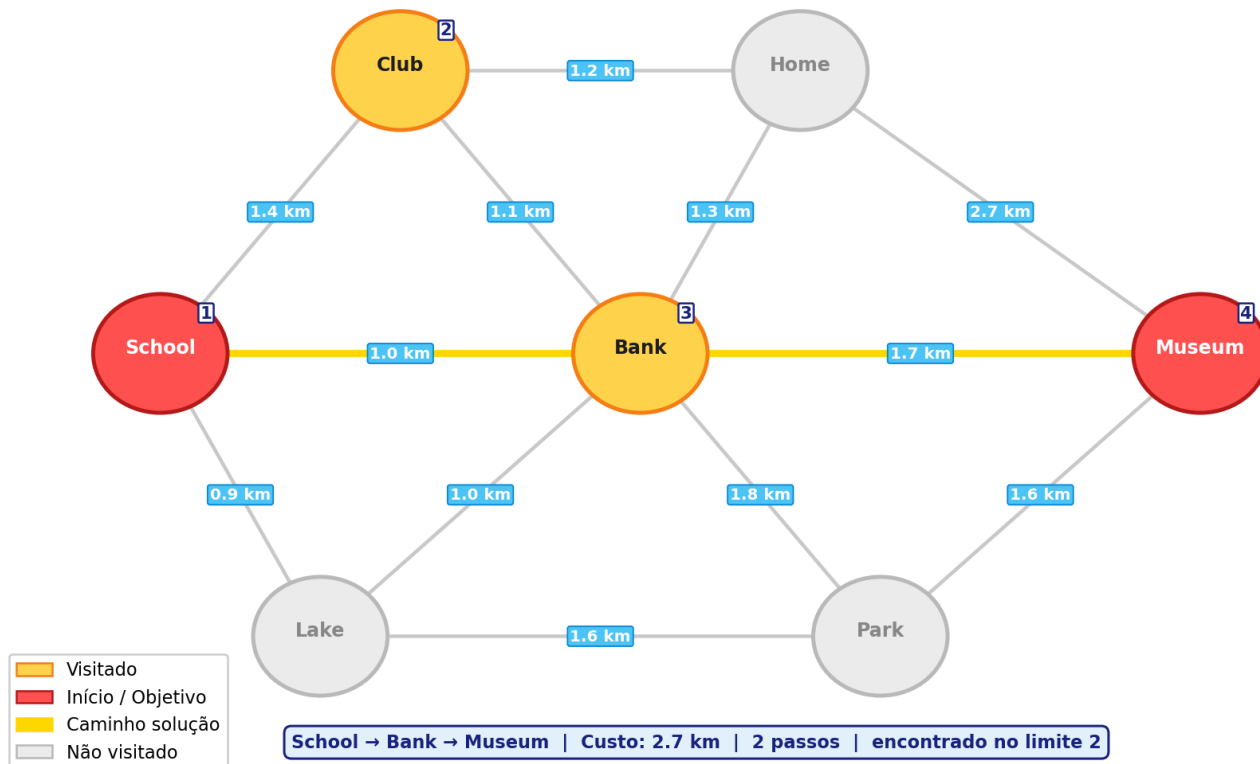
Caminho: School -> Bank -> Museum

Custo: 2.7 km

Passos: 2

segue o exemplo:

Profundidade Iterativa (IDS, limite=4) — School → Museum
Solução encontrada na iteração limite = 2



8. Execute os algoritmos de busca em Largura e Profundidade em nível de código na IDE de sua preferência para o mapa da questão anterior. Colete os tempo de execução de cada algoritmo e diga qual foi o mais rápido.

```
leo@leo-ASUS-TUF-Gaming-A16-FA607NUG-FA607NUG:~/Área de trabalho/UTFPR/Fundamentos de IA/Listas de exercicios/respostas/
BFS - Busca em Largura
Caminho: School -> Bank -> Museum
Custo total: 2.7 km
Nos visitados: 6
Nos expandidos: 5
Tempo: 0.000011 segundos

DFS - Busca em Profundidade
Caminho: School -> Club -> Home -> Bank -> Museum
Custo total: 5.6 km
Nos visitados: 5
Nos expandidos: 4
Tempo: 0.000010 segundos

RESULTADO
Mais rapido: DFS
Menor custo: BFS (2.7 km)
```

DFS é mais rápido em tempo mas encontra caminho mais longo.

9. Altere os algoritmos para criar o algoritmo de busca Profundidade limitada e Aprofundamento Iterativo e execute eles no mapa da Romênia (considera apenas as cidades que estão atrás de Bucharest) para sair de Arad e chegar em Bucarest. Teste os

algoritmos com limites 2, 4 e 7.

```
DLS - Busca em Profundidade
DLS (limite=2): Nenhum caminho encontrado
DLS (limite=4): Arad -> Sibiu -> Fagaras -> Bucharest
                Custo: 450 km
DLS (limite=7): Arad -> Zerind -> Oradea -> Sibiu -> Fagaras -> Bucharest
                Custo: 607 km

Tempo total DLS (3 execucoes): 0.000033 segundos

IDS - Aprofundamento
IDS (ate limite=2): Nenhum caminho encontrado
IDS (ate limite=4): sucesso no limite 3
    Caminho: Arad -> Sibiu -> Fagaras -> Bucharest
    Custo:    450 km
IDS (ate limite=7): sucesso no limite 3
    Caminho: Arad -> Sibiu -> Fagaras -> Bucharest
    Custo:    450 km

Tempo total IDS 3 execucoes: 0.000040 segundos
```

OBSERVAÇÕES

- DLS com limite=2: muito raso, nao alcanca Bucharest com distância 3.
- DLS com limite=4: encontra Arad->Sibiu->Fagaras->Bucharest tendo 450 km.
- DLS com limite=7: encontra caminho mais longo via Zerind com 607 km,pois DFS explora esse ramo antes e retorna a primeira solução válida.
- IDS sempre encontra o caminho mais raso independente do limite máximo, pois itera de 0 ate achar.